PUC Minas

CarFlow

Sistema de Aluguel de Carros

Laboratório de Desenvolvimento de Software - 2026.1

Laura · Eric · Giuliano

Integrantes do Grupo

PUC Minas · Laboratório de Desenvolvimento de Software II

Eric Gregório

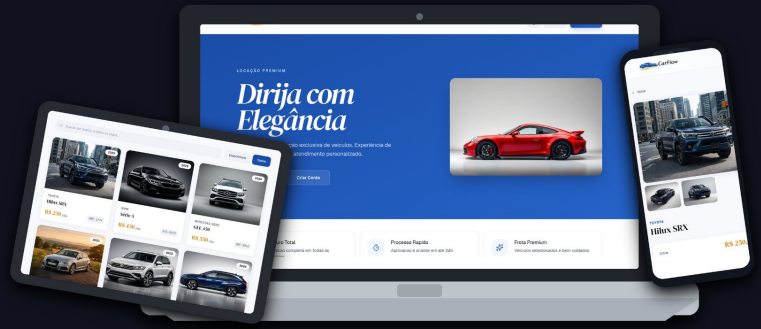
Giuliano Percope

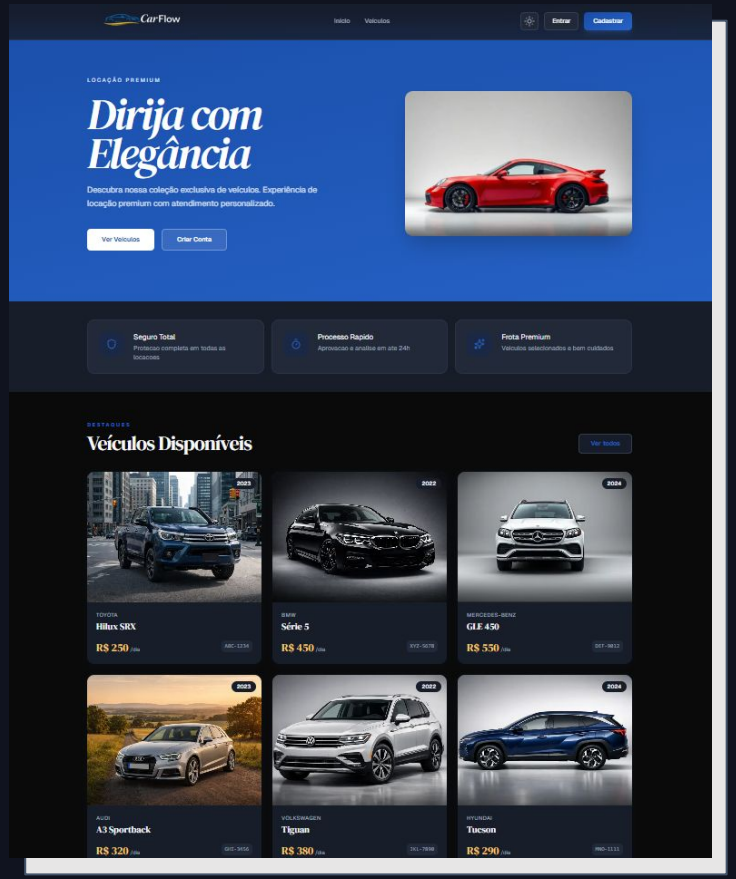
Laura Pontara

O Projeto

Objetivo

Desenvolver um sistema web completo para gerenciamento de aluguel de automóveis, aplicando arquitetura de microsserviços com Micronaut



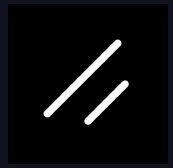


Tecnologias utilizadas

Backend microsserviços

Micronaut 4

Frontend



Next.js + TypeScript + Tailwind + Shadcn

Orquestração local

Docker

Deploy produção

Vercel + Render

Banco de Dados

PostgreSQL

Histórias de Usuário

Cliente

- Cadastro com RG, CPF, profissão e entidades empregadoras
- Consulta de automóveis disponíveis para locação
- Criação e manutenção de pedidos em rascunho
- Envio de pedido para análise
- Cancelamento de pedidos em análise
- Histórico de locações realizadas
- Validação automática de financiamento por renda
- Notificação de aprovação do pedido

Agente Locador

- Cadastro como empresa, banco ou locador autônomo
- Cadastro de automóveis (matrícula, ano, marca, modelo, placa)
- Visualização de pedidos pendentes dos seus veículos
- Análise e parecer em pedidos de locação
- Dashboard de histórico de pedidos analisados
- Registro de propriedade do automóvel alugado

Agente Banco

- Cadastro como banco agente no sistema
- Visualização de pedidos aprovados aguardando financiamento
- Associação de contrato de crédito ao aluguel
- Emissão de parecer de aprovação/reprovação de financiamento

Total: 19 Histórias de Usuário

Diagrama de Caso de Uso

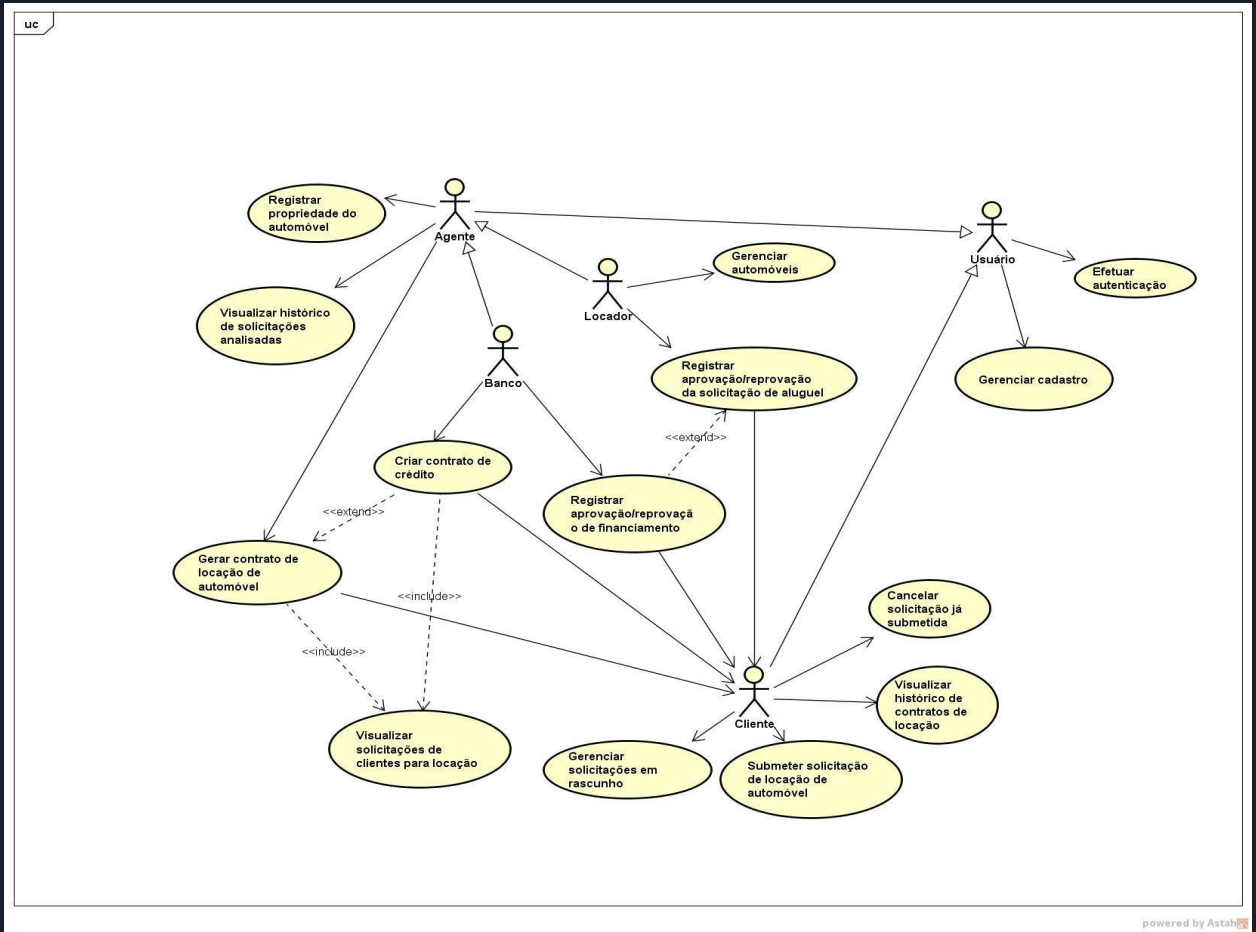
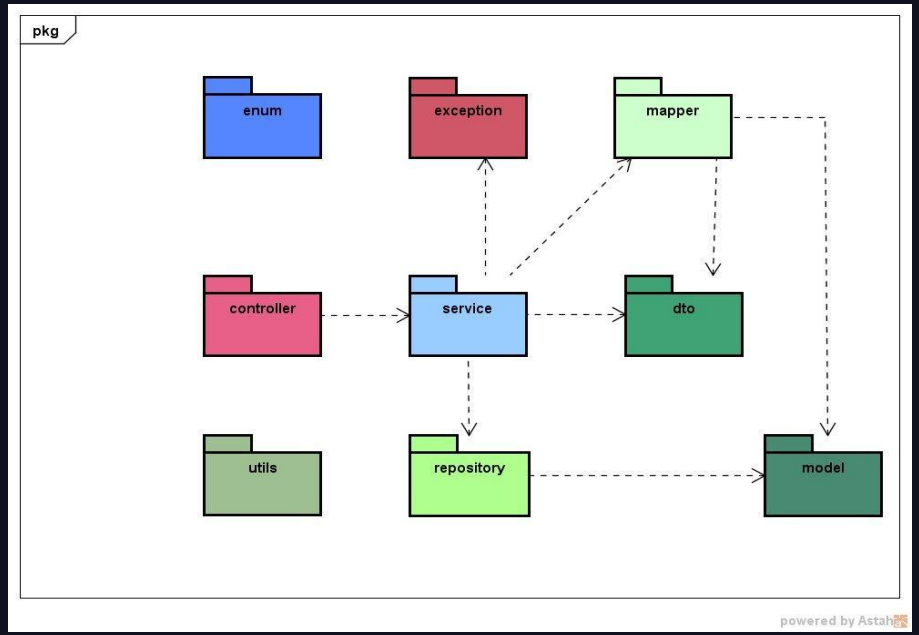


Diagrama de Pacotes

Organização interna de cada microsserviço — padrão seguido em todos os serviços



controller	Recebe requisições HTTP e delega ao service
service	Lógica de negócio, orquestração e regras
repository	Acesso ao banco via Micronaut Data JDBC
model	Entidades mapeadas com @MappedEntity
dto	Objetos de transferência de dados (req/resp)
mapper	Conversão model ↔ DTO via MapStruct
exception	Handlers de erro HTTP padronizados
enums	StatusGeral, StatusLocador, TipoAgente…

Diagrama de Classes

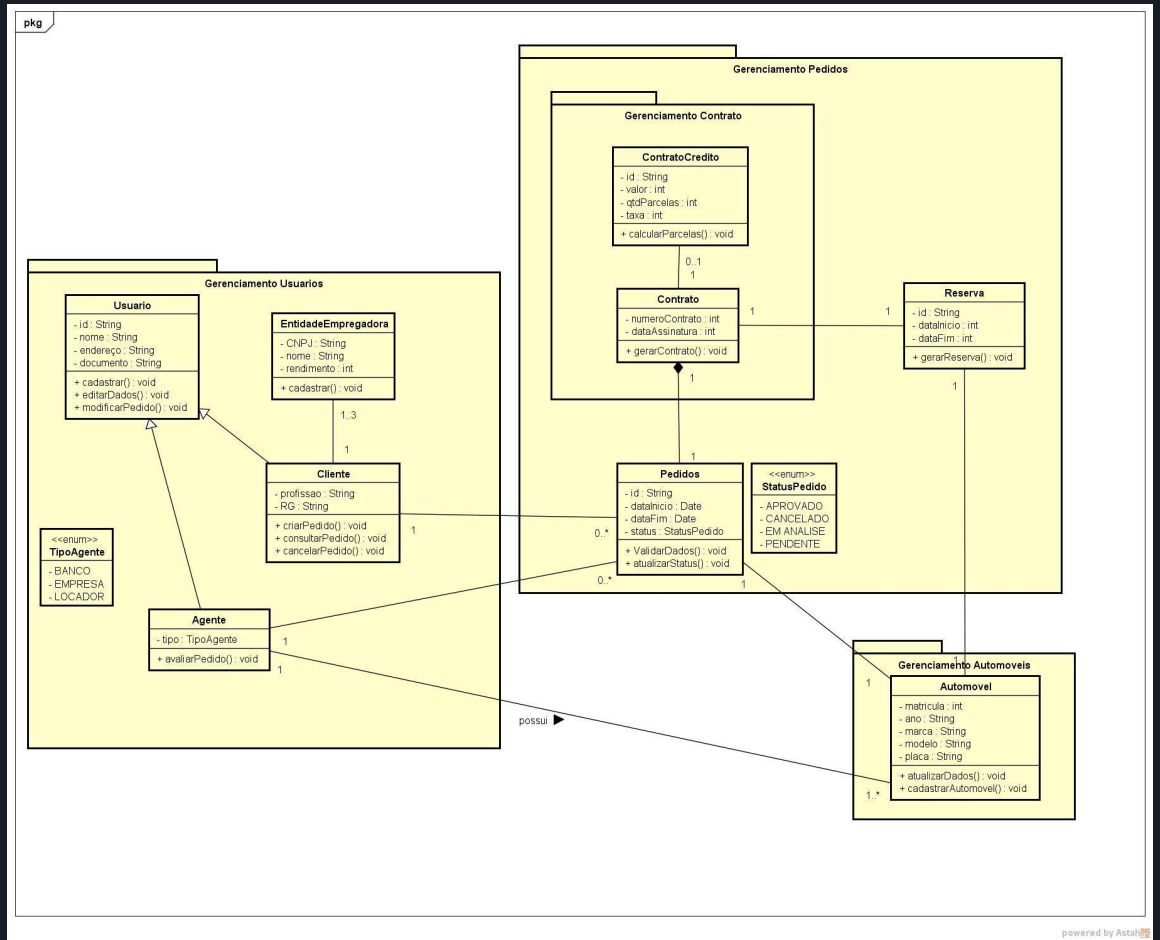


Diagrama de Componentes

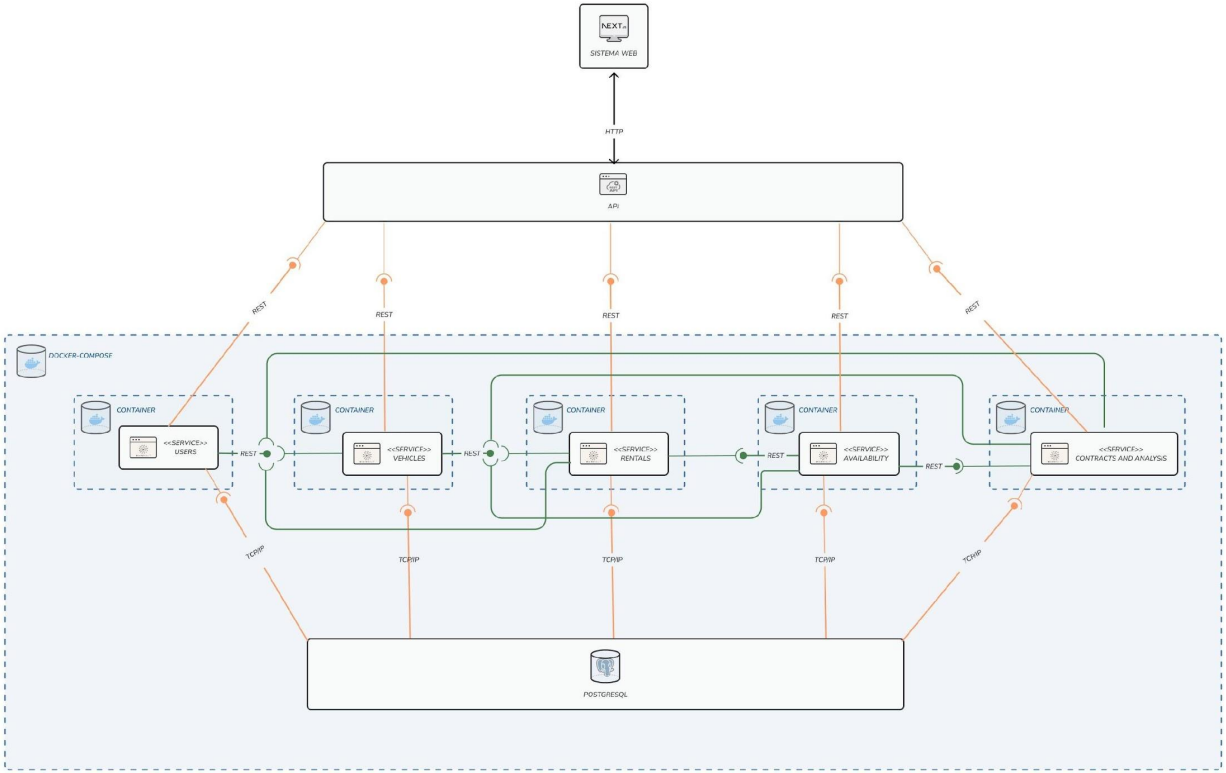


Diagrama de Implantação

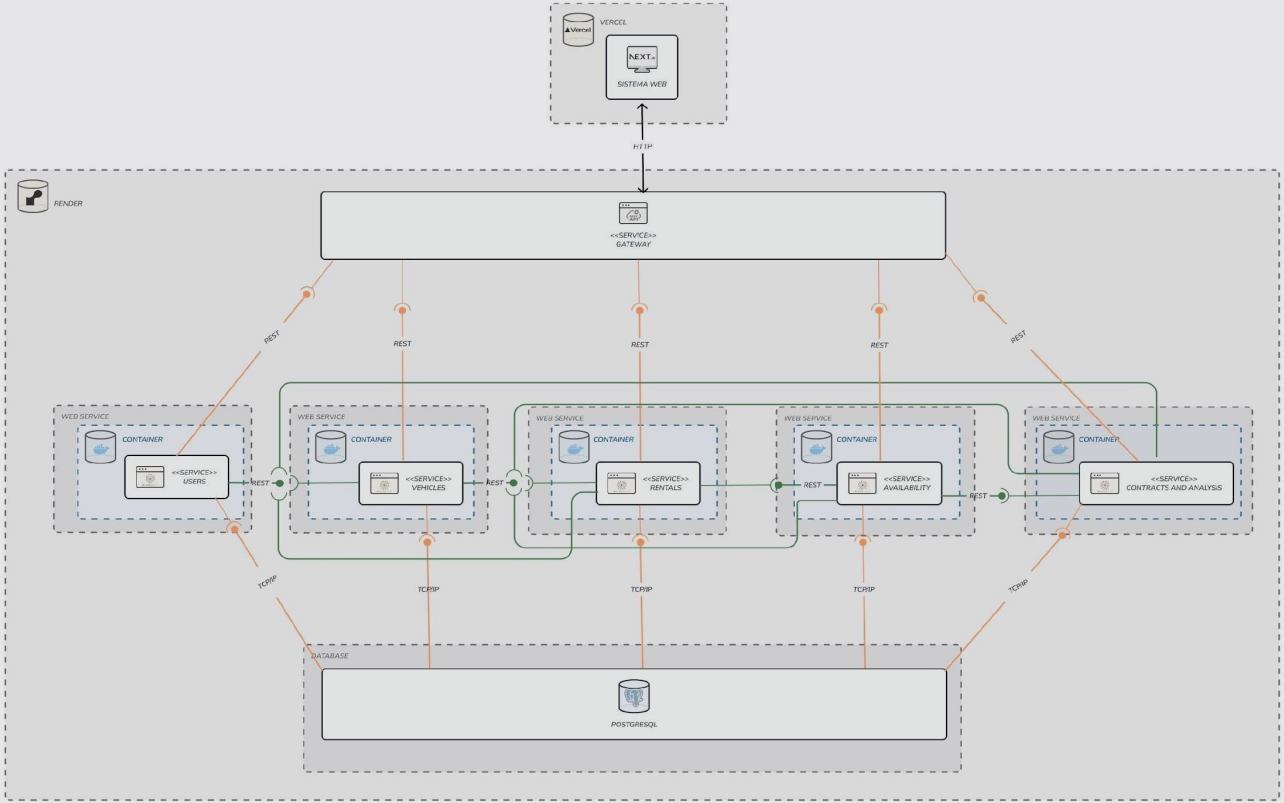
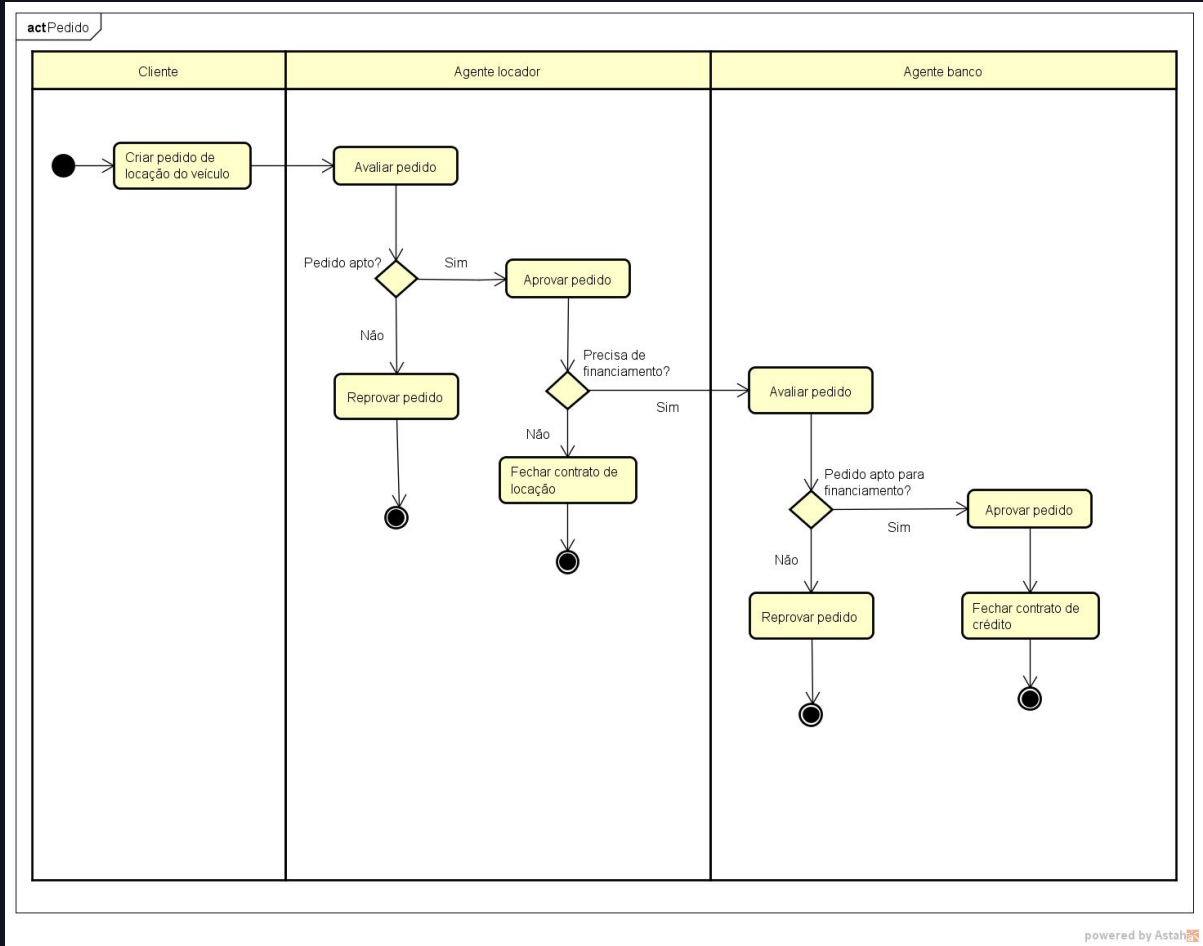


Diagrama de atividades



Os 5 Microsserviços

API Gateway · porta 8000

:8080

usersService

Gerencia clientes, agentes e autenticação JWT. Cadastro com entidades empregadoras, renda total e tipos de agente (Empresa, Banco, Locador).

:8081

vehiclesService

Cadastro e gerenciamento de automóveis. Controla locador original, proprietário atual e disponibilidade. Endpoint interno para contractsService.

:8082

rentalsService

Ciclo de vida dos pedidos de locação: RASCUNHO → SUBMETIDO → EM_ANALISE_BANCO → CONTRATO_FECHADO. Integra com usersService e vehiclesService.

:8083

contratoService

Formalização de contratos de locação e crédito. Gera contratos, associa banco ao financiamento e fornece dashboard de métricas para agentes.

:8084

reservasService

Gerencia o calendário de disponibilidade dos veículos, verificação de conflitos de datas e bloqueio de períodos de reserva.

Nossas Percepções

O que aprendemos usando Micronaut no projeto

✅ Pontos Positivos

- Tempo de startup rápido (ideal para microsserviços)
- Injeção de dependência em compile-time pegou erros mais cedo
- @Client declarativo tornou chamadas inter-serviço muito limpas
- Baixo consumo de memória por container
- Integração com Render e JDBC simples e previsível

⚠️ Desafios Encontrados

- Curva de aprendizado maior (menos tutoriais que Spring Boot)
- Mensagens de erro de compile-time menos descritivas às vezes
- Configuração inicial mais complexa (exige mais ajustes manuais em comparação ao Spring Boot)
- Menor produtividade inicial (mais tempo para estruturar o projeto)
- Ecossistema menor, algumas libs não têm suporte nativo

💡 Conclusão

- Para microsserviços: Micronaut é superior ao Spring Boot em performance
- Para equipes com background Spring: curva de adaptação de (1 semana)
- Arquitetura AOT força boas práticas desde o início
- Spring Boot ainda vence em ecossistema e comunidade

Obrigado!

Laura · Eric · Giuliano

PUC Minas — Laboratório de Desenvolvimento de Software II — 2026

lab-carflow.vercel.app